Assignment 6 Visitor Pattern

Software Design Patterns | Advanced File System | Java 17 and Maven

Amirkhan Tulkibayev
Group SE-2522

https://github.com/hann-qaz/Assignment6_visitor_pattern

Purpose and structure

The project uses Visitor to run three operations on one in-memory file system: total size, filename search and tree statistics. FileNode stores a name and byte size. DirectoryNode stores named children. Node holds the shared name and parent reference; Element and Visitor define the pattern contracts.

Double dispatch and traversal

First, scan calls root.accept(this) through Element, so Java selects the concrete element implementation. Second, FileNode.accept calls visitor.visit(this), where this is a FileNode. This selects visit(FileNode); the visitor object supplies the operation. DirectoryNode works the same way with visit(DirectoryNode). No type-checking chain is needed.

Visitors own the recursive traversal. A directory visit calls child.accept(this) once per child; accept itself does not walk. TotalSizeVisitor adds only file sizes. SearchVisitor combines a Predicate<String> with an inclusive minimum size. StatisticsVisitor counts every file and directory, including a directory root.

Search paths start at the chosen root, omit its directory name, use / and are sorted lexicographically. A file root uses its own name. Depth counts edges: the chosen root is 0, its children are 1. Empty directories also affect maximum depth.

Demonstration

Main.createDemoTree() builds notes.txt (120), documents/notes.txt (340), documents/plan.pdf (800), documents/archive/old.txt (60), images/logo.png (2048), and an empty directory. The observed default output is:

```
Root: workspace
Total size: 3368 bytes
Search: name contains '.txt', size >= 120
  documents/notes.txt
  notes.txt
Files: 5
Directories: 5
Maximum depth: 3
```

Integrity and visitor state

DirectoryNode.addChild checks duplicate sibling names, existing ownership and ancestor cycles before changing children or parent. Names and sizes cannot be edited. Negative sizes and invalid names are rejected with IllegalArgumentException. getChildren returns an immutable snapshot. TotalSizeVisitor uses Math.addExact to reject long overflow.

Start every operation with scan(root), which clears previous results, paths and depth. Visitors can be reused sequentially; direct accept calls are traversal steps. Read results after a successful scan. Search results are immutable snapshots. The project does not support concurrent scans or mutations during traversal.

Five Clean Code practices

Meaningful naming. TotalSizeVisitor.getTotalSize and StatisticsVisitor.getMaximumDepth state exactly what their results mean.

Single Responsibility. DirectoryNode manages tree integrity, while each visitor implements one operation.

Small methods. FileNode.accept and DirectoryNode.accept each contain one dispatch call, so the pattern is easy to trace.

Encapsulation. DirectoryNode.getChildren uses List.copyOf, preventing callers from bypassing addChild checks.

No type-checking chains. The two accept methods call the matching Visitor.visit overload without instanceof, reflection or casts.

Automated verification

Run mvn test from the project root. Verified on Eclipse Temurin JDK 17.0.21 and Maven 3.9.11 on 8 November 2026. Actual result: 25 tests, 0 failures, 0 errors, 0 skipped; BUILD SUCCESS.

Test class	Tests	Main checks
DoubleDispatchTest	2	Both overloads; accept does not walk
VisitorsTest	10	Size, depth, repeat runs, order, edge cases
SearchVisitorTest	7	Combined filters, paths, sorting, snapshots
TreeIntegrityTest	6	Invalid changes, ownership, cycles, snapshots

Extension and defense

StatisticsVisitor was added in commit 52f529b after c53bd9b, without changing FileNode, DirectoryNode or Visitor. A new operation needs another visitor. A new element type needs an extra Visitor overload and changes in all visitors. Visitor traversal repeats a small child loop, but keeps this example easy to follow.

Run mvn compile exec:java -Dexec.args=".pdf 500" to change the search: only documents/plan.pdf matches. Trace scan -> accept -> visit, then explain why 120 bytes meets the default inclusive threshold. The code follows Lecture 9, especially pages 3, 18 and 20.

UML class diagram

The elements dispatch calls; visitors contain operations and recursive traversal. The parent association allows at most one owner for each child.

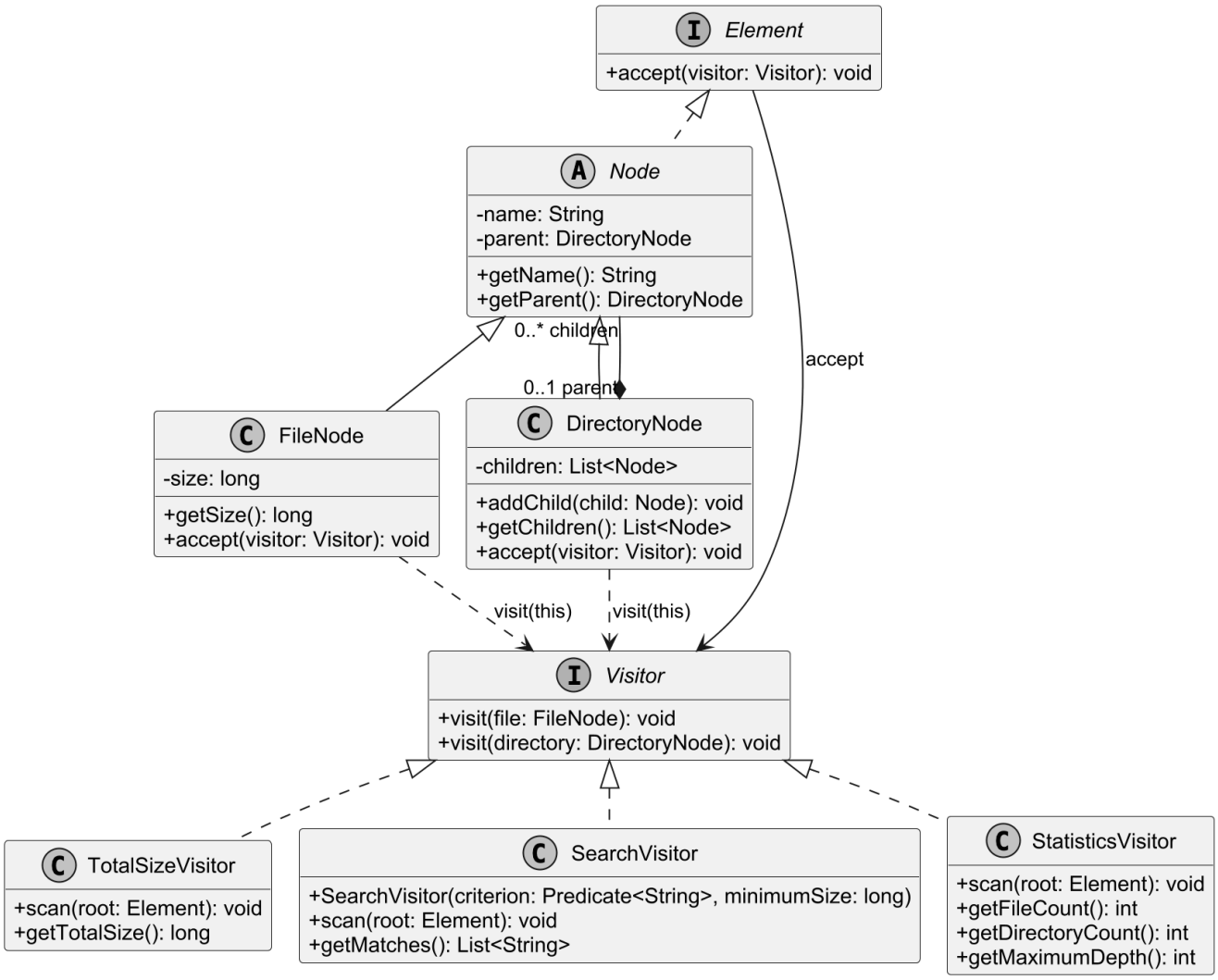


Figure 1. Element hierarchy and the three visitors. Concrete visitors implement both visit overloads declared by Visitor; these inherited contracts are omitted from their boxes for readability.

Dispatch example

```
Element file = new FileNode("notes.txt", 120);
file.accept(sizeVisitor);
// FileNode.accept calls sizeVisitor.visit(this).
```

The editable class and sequence diagrams are provided as PlantUML sources with SVG previews in docs. The sequence diagram traces the same two dispatch steps through nested directories.